



Reinforcement Learning from Human Feedback

(RLHF)

How human preferences shape AI behavior

Presentation generated by Claude Sonnet – May 2026

What is RLHF?

RLHF is a machine learning technique that trains AI models using feedback from humans rather than handcrafted reward functions — aligning AI behavior with human values and preferences.

Reinforcement Learning

Agent learns by trial and error, optimizing a reward signal over time

Human Feedback

Humans rate or compare model outputs to express preferences

Fine-Tuned Model

A reward model captures human preferences; the LLM is optimized against it

Why RLHF?

✗ Challenges Without RLHF

- Hard to define reward functions for complex tasks
- Models optimize for the wrong objective (reward hacking)
- Generated text may be harmful, biased, or unhelpful
- No mechanism to encode nuanced human values

✓ RLHF Benefits

- Captures complex, hard-to-specify preferences
- Naturally discourages harmful or misleading outputs
- Iteratively improves with ongoing human signal
- Enables alignment with diverse human values

The RLHF Pipeline

Three stages to align a language model with human preferences

01

Supervised Fine-Tuning

Pre-trained model learns from curated human-written demonstrations to establish a quality baseline.

02

Reward Model Training

Human raters compare pairs of outputs. A reward model learns to predict human preference scores.

03

RL Optimization (PPO)

The LLM is fine-tuned via PPO, maximizing the reward model's score while staying close to the SFT baseline.

Stage 1 — Supervised Fine-Tuning (SFT)

How It Works

- 1 Start with a large pre-trained language model (e.g., GPT)
- 2 Collect a dataset of high-quality prompt-response pairs written by humans
- 3 Fine-tune the model using standard supervised learning (cross-entropy loss)
- 4 Result: a model that broadly follows instructions

Key Insight

SFT gives the model a strong starting point. Without it, reinforcement learning would explore from scratch — an inefficient and unstable process.

The SFT baseline defines the initial policy π_0 that PPO will later optimize.

Output: SFT Model (π_{SFT})

Stage 2 — Reward Model Training

Human Comparison Process

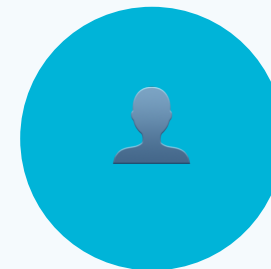
Prompt

Response A

"Here is a helpful and safe answer..."

Response B

"Here is a less aligned answer..."



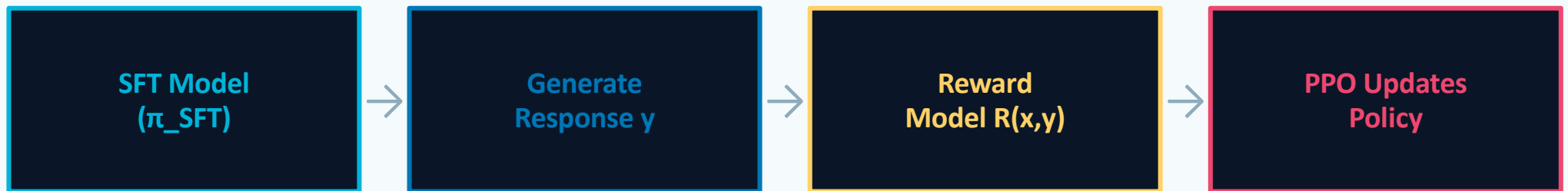
**Human Rater
Prefers A**

**Reward
Model
RM(x,y)**

The reward model is trained to assign higher scores to responses humans prefer — encoding their values in a differentiable function.

Stage 3 — RL Optimization via PPO

Proximal Policy Optimization (PPO)



KL Divergence Penalty

To prevent the model from drifting too far from the SFT baseline, PPO adds a KL-divergence penalty:

$$\text{Objective} = R(x,y) - \beta \cdot \text{KL}(\pi \parallel \pi_{\text{SFT}})$$

This keeps responses coherent while maximizing reward.

Result: RLHF Model

- Follows instructions reliably
- Refuses harmful requests
- Provides helpful, accurate answers
- Aligns with human values

DPO Deep Dive — Direct Preference Optimization

How DPO Differs from PPO

PPO Approach

1. Train a separate Reward Model
2. Run RL (PPO) against that model
3. Needs multiple components:
SFT model, RM, policy, value fn
4. Computationally expensive
5. Unstable — sensitive to hyperparams

DPO Approach

1. No separate Reward Model needed
2. Optimize LM directly on preferences
3. Only needs: SFT model + preference data
4. Much simpler, faster to train
5. Mathematically equivalent to PPO*

DPO Loss Function

$$L_{DPO} = -E[\log \sigma(\beta \cdot \log(\pi_{\theta}(y_w|x)/\pi_{ref}(y_w|x)) - \beta \cdot \log(\pi_{\theta}(y_l|x)/\pi_{ref}(y_l|x)))]$$

y_w = preferred (winning) response | y_l = dispreferred (losing) response | β = temperature | π_{ref} = reference (SFT) policy

DPO — Intuition & How It Works

Intuition: DPO implicitly defines a reward function inside the language model itself. By training on (preferred, rejected) pairs, it directly nudges the model to assign higher probability to preferred responses — without ever computing an explicit reward score.

1

Collect Preference Pairs

For each prompt x , collect a preferred response y_w and a rejected response y_l from human raters (same data as RLHF).

2

Compute Log Ratios

For both responses, compute how much more (or less) likely the policy is to generate them compared to the reference SFT model.

3

Maximize Margin

Optimize so the log-ratio gap between preferred and rejected responses grows larger — i.e. preferred responses become more likely relative to the SFT baseline.

4

β Controls Deviation

β (temperature) controls how far the model can drift from the SFT policy. Low β = stay close; high β = larger updates allowed.

PPO vs DPO — Side-by-Side Comparison

Aspect	PPO	DPO
Approach	On-policy RL with a learned reward model	Direct supervised loss on preference pairs
Reward Model	Required — separate model trained on comparisons	Not needed — reward is implicit in the LM
Complexity	High — multiple models & RL loop	Low — single model, standard fine-tuning
Stability	Can be unstable, sensitive to hyperparams	More stable, easier to tune
Sample Efficiency	Generates new data online (costly)	Works on fixed offline preference dataset
Performance	State-of-the-art, especially on complex tasks	Competitive; may lag on hard reasoning
Used By	OpenAI (ChatGPT), Anthropic, DeepMind	Mistral, Llama fine-tunes, many open-source LLMs

Real-World Applications



ChatGPT / GPT-4

OpenAI used RLHF to make GPT models helpful, harmless, and honest in conversation.



Claude (Anthropic)

Anthropic trains Claude with RLHF and Constitutional AI to align with human values.



Gemini (Google)

Google DeepMind applies RLHF to improve instruction-following and factual grounding.



Code Generation

GitHub Copilot and similar tools use RLHF to produce more useful, less buggy code.



Game AI

RLHF trains game agents that play in more human-like, engaging ways.



Healthcare AI

Medical AI assistants use RLHF to communicate safely and defer appropriately to experts.

Challenges & Limitations



High Cost

Collecting large-scale, quality human preference data is expensive and time-consuming.



Annotator Bias

Human raters have differing values, cultural perspectives, and biases that can influence the reward model.



Reward Hacking

Models may find ways to score high on the reward model without actually being helpful or honest.



Scalability

As tasks grow in complexity, human feedback becomes harder to gather reliably and at scale.



Distribution Shift

The reward model may not generalize to novel prompts that weren't in the training data.



Goodhart's Law

"When a measure becomes a target, it ceases to be a good measure." Over-optimization degrades quality.

Beyond RLHF — Future Directions

Constitutional AI (CAI)

Anthropic's approach: the model critiques its own outputs using a written "constitution" — reducing reliance on human labels.

RLAIF

Reinforcement Learning from AI Feedback uses an AI instead of humans to label preferences — scalable but requires careful validation.

Direct Preference Optimization (DPO)

A simpler method that directly optimizes the language model on preference data, bypassing the separate reward model entirely.

Process Reward Models

Instead of just rewarding final outputs, reward each reasoning step — improving accuracy on complex multi-step tasks.

Key Takeaways



RLHF is a 3-stage process: SFT → Reward Modeling → PPO optimization



PPO clips policy updates (ϵ) and adds KL penalty to prevent unstable training



DPO simplifies RLHF: no reward model needed — preferences shape the LM directly



PPO excels on complex tasks; DPO is faster, simpler, and widely used in open-source



Key challenges include cost, bias, reward hacking, and scalability



Newer methods (CAI, RLAIF, Process Rewards) push alignment even further

RLHF: Making AI that humans actually want to use.

Policy Gradient Algorithms

A2C / A3C vs PPO vs TRPO

A comparative overview of on-policy deep reinforcement learning

A2C

A3C

PPO

TRPO

A2C & A3C — Advantage Actor-Critic

Slide 2 / 4

A3C Asynchronous Advantage Actor-Critic (2016)

Core Idea

Multiple workers run in parallel threads, each with its own environment copy. Workers compute gradients asynchronously and push updates to a shared global network.

✓ Strengths

- No replay buffer needed
- Naturally diverse experience
- Fast wall-clock time on CPU

✗ Weaknesses

- Asynchronous updates → stale gradients
- Hard to reproduce exactly
- Inconsistent policy during training

$$\nabla J \approx A(s,a) \cdot \nabla \log \pi(a|s) \quad \text{where} \quad A = r + \gamma V(s') - V(s)$$

A2C Synchronous Advantage Actor-Critic (2016)

Core Idea

Synchronised version of A3C. All parallel workers step together; gradients are averaged before the single update. Cleaner, more stable training.

✓ Strengths

- No stale gradients
- Easier to implement & debug
- GPU-friendly (batched update)

✗ Weaknesses

- Slower — waits for slowest worker
- Still no trust region
- High variance gradients

$$\text{Update} = \text{mean_workers}[A(s,a) \cdot \nabla \log \pi(a|s)]$$

⚠ Both A2C & A3C share a key limitation: no constraint on how large each policy update can be → training can destabilise

TRPO & PPO — Trust Region Methods

Slide 3 / 4

TRPO Trust Region Policy Optimisation (2015)

Core Idea

Constrains the KL divergence between old and new policy to stay within a "trust region". Solves a constrained optimisation problem using conjugate gradient + line search.

✓ Strengths

- Monotonic policy improvement guarantee
- Very stable updates
- Strong theoretical foundation

✗ Weaknesses

- Computationally expensive (2nd-order)
- Complex to implement
- Doesn't scale well to large networks

$$\max L(\theta) \quad \text{s.t.} \quad E[\text{KL}(\pi_{\text{old}} \parallel \pi_{\text{new}})] \leq \delta$$

PPO Proximal Policy Optimisation (2017)

Core Idea

Approximates TRPO's constraint with a simple clipped surrogate objective. No second-order optimisation needed — just clipping the probability ratio $r(\theta) = \pi_{\text{new}} / \pi_{\text{old}}$.

✓ Strengths

- Simple & fast (first-order)
- Excellent empirical performance
- Easy to tune & parallelise

✗ Weaknesses

- Clipping is a heuristic (no hard guarantee)
- Sensitive to clip parameter ϵ
- Can still be unstable at scale

$$L_{\text{CLIP}} = E[\min(r \cdot A, \text{clip}(r, 1-\epsilon, 1+\epsilon) \cdot A)]$$

💡 PPO is essentially TRPO made practical: same motivation, ~10× simpler code, nearly identical empirical performance

Head-to-Head Comparison

Slide 4 / 4

Property	A3C	A2C	TRPO	PPO
Update type	Async, parallel	Sync, parallel	Constrained optim.	Clipped surrogate
Gradient order	1st order	1st order	2nd order	1st order
Policy guarantee	None	None	Monotonic improv.	Heuristic
Complexity	Medium	Low	High	Low
Sample efficiency	Medium	Medium	Medium	Medium-High
Scalability	CPU-friendly	GPU-friendly	Poor (large nets)	Excellent
Typical use	Classic games	Classic games	Research/robotics	General purpose

Use A2C:

simple baselines, fast iteration

Use TRPO:

safety-critical, need guarantees

Use PPO:

most tasks — it's the default choice

Continuous Control

Deep RL Algorithms

DDPG · TD3 · SAC — off-policy methods for continuous action spaces

DDPG

TD3

SAC

Off-Policy

Model-Free

Continuous Actions

Actor-Critic

Lillicrap et al. (2015) · Fujimoto et al. (2018) · Haarnoja et al. (2018)

DDPG — Deep Deterministic Policy Gradient

Slide 2 / 4

DDPG

Lillicrap et al., 2015

Core Idea

Extends DQN to continuous action spaces using a deterministic actor. Combines the actor-critic framework with a replay buffer and target networks. Actor outputs a single action (not a distribution); critic evaluates it.

✓ Strengths

- First off-policy continuous-action method
- Deterministic policy → low variance updates
- Sample efficient via replay buffer

✗ Weaknesses

- Q-value overestimation bias
- Very sensitive to hyperparameters
- Brittle training; can diverge suddenly

$$a = \mu(s|\theta) \quad Q\text{-loss: } (r + \gamma Q(s', \mu(s')) - Q(s, a))^2$$

Architecture

Actor $\mu(s)$

Deterministic policy

Target Actor $\mu'(s)$

Soft-updated copy

Critic $Q(s,a)$

Evaluates (state, action)

Target Critic $Q'(s,a)$

Soft-updated copy

Replay Buffer

Off-policy experience

Exploration: Ornstein-Uhlenbeck noise added to deterministic action at training time

TD3 & SAC — Fixes and Extensions to DDPG

Slide 3 / 4

TD3 Twin Delayed Deep Deterministic (2018)

Core Idea

Directly addresses DDPG's overestimation problem with three key tricks: twin critics (take the min), delayed policy updates (actor updates less often than critic), and target policy smoothing (add noise to target actions).

✓ Strengths

- Twin Q reduces overestimation
- Delayed updates stabilise training
- More robust than DDPG

✗ Weaknesses

- Still deterministic — limited exploration
- Requires careful noise tuning
- Hyperparameter sensitive

$$y = r + \gamma \cdot \min(Q_1', Q_2')(s', \tilde{a}) \quad \tilde{a} = \mu'(s') + \text{clip}(\epsilon, -c, c)$$

SAC Soft Actor-Critic (2018)

Core Idea

Replaces deterministic policy with a stochastic one and adds entropy maximisation to the objective. Uses twin critics (from TD3) and auto-tunes the temperature α . Encourages exploration intrinsically — no manual noise needed.

✓ Strengths

- Best sample efficiency in class
- Auto-tuned entropy (α)
- Robust — minimal hyperparameter tuning

✗ Weaknesses

- More complex than TD3
- Stochastic policy has higher variance
- Entropy target needs rough calibration

$$\mathcal{J}(\pi) = \sum E[r(s, a) + \alpha \cdot H(\pi(\cdot | s))] \quad \alpha \text{ auto-tuned}$$

🔗 Lineage: DQN → DDPG → TD3 (fixes overestimation) · DDPG + entropy maximisation → SAC

Head-to-Head Comparison

Slide 4 / 4

Property	DDPG	TD3	SAC
Policy type	Deterministic	Deterministic	Stochastic
Overestimation fix	None	Twin Q (min)	Twin Q (min)
Exploration	OU noise (manual)	Target noise (manual)	Entropy bonus (auto)
Temperature α	N/A	N/A	Auto-tuned
Stability	Brittle	Moderate	High
Sample efficiency	Medium	Medium-High	High
Hyperparameter sensitivity	Very High	Medium	Low
Typical use	Baseline / legacy	Continuous control	State-of-the-art

Use DDPG:

legacy comparison / simple envs

Use TD3:

deterministic, well-tuned control tasks

Use SAC:

default choice — most robust & efficient